APPLICATION NOTE

How to Run Basler Cameras in a Docker Container

Applicable to
Basler USB3 Vision cameras or Basler dart BCON for MIPI camera modules only

Document Number: AW001730
Version: 01 Language: 000 (English)
Release Date: 25 May 2022

BASLER
the power of sight

Contacting Basler Support Worldwide

Europe, Middle East, Africa

Basler AG
An der Strusbek 60–62
22926 Ahrensburg
Germany

Tel. +49 4102 463 515
Fax +49 4102 463 599

support.europe@baslerweb.com

The Americas

Basler, Inc.
855 Springdale Drive, Suite 203
Exton, PA 19341
USA

Tel. +1 610 280 0171
Fax +1 610 280 7608

support.usa@baslerweb.com

Asia-Pacific

Basler Asia Pte. Ltd.
35 Marsiling Industrial Estate Road 3
#05–06
Singapore 739257

Tel. +65 6367 1355
Fax +65 6367 1255

support.asia@baslerweb.com

www.baslerweb.com

All material in this publication is subject to change without notice and is copyright Basler AG.

Table of Contents

1 General Information **2**

2 Creating a Dockerfile **2**

3 Creating a Docker Image **4**

4 Running the Docker Container **5**

1 General Information

This application note describes how to run a Basler camera in a Docker container.

A Docker container can be regarded as a kind of lightweight virtual machine (VM). In contrast to a VM, a Docker container uses the kernel of the host system, which means that the operating system that runs inside the Docker container must be the same as the host system's operating system.

The procedures described in this document assume that you are using a Linux host system, e.g., Ubuntu.

2 Creating a Dockerfile

In order to run a Docker container, you first need to create a Docker image that can be used by the container. A Docker image is described by a Dockerfile. You do not need to create an image from scratch. Instead, you can derive your image from an already existing image. Plenty of existing images are available at Docker Hub. If you, e.g., derive your Docker image from an Ubuntu image, your image will then contain all basic Ubuntu features (e.g., aptitude, which makes it possible to install additional Debian packages into your container).

Here's an example of a Dockerfile, that would create a Docker image that already contains everything you need to run a Basler camera in your container:

```
# Derive your docker image from Ubuntu 21.10

FROM ubuntu:21.10


# In case that some of the following steps for installing Debian packages

# require user interaction, this is now disabled and all is answered with yes

ARG DEBIAN_FRONTEND=noninteractive
```

```
# your package manager database will be updated and some additional useful
# packages will be installed. E.g., qtbase5-dev is required to run the pylon
# viewer in the container
RUN apt-get update && \
    apt-get install -y --no-install-recommends ca-certificates wget tar g++ make \
    libgl1-mesa-glx v4l-utils bzip2 strace less qtbase5-dev iproute2 nano

# The pylon SDK Debian package is downloaded…
RUN wget \
https://www.baslerweb.com/fp-1594648231/media/downloads/software/pylon_software/pylon_6.2.0.21487-deb0_arm64.deb && \
# …and installed
dpkg -i pylon_6.2.0.21487-deb0_arm64.deb

# If you also want to run mipi cameras in your container, you need to copy the
# mipi GenTL producers (*.cti files) into the container to the right location
# (/opt/dart-bcon-mipi/lib). Make sure that the cti files fit to the pylon version
# which you pylon to install in the container and that they are placed in your
# current directory
COPY *.cti /opt/dart-bcon-mipi/lib/
```

```
# A basic pylon C++ sample (Grab) will be compiled and placed into the container's
# home directory - it allows for running the Grab program immediately after the
# container has been started
RUN cd /opt/pylon/share/pylon/Samples/C++/Grab && \
    make &&\
    cp Grab /home/

# An alias "pylon" for the pylon viewer is created
RUN echo 'alias pylon="/opt/pylon/bin/pylonviewer -platform xcb"' >> ~/.bashrc
# The GENICAM_GENTL64_PATH environment variable is set - this variable allows
# the system to find the mipi GenTL producers. This is only needed if you
# want to run Basler's mipi cameras
ENV GENICAM_GENTL64_PATH="/opt/dart-bcon-mipi/lib/"

# The container shell starts in its home directory
WORKDIR /home
```

3 Creating a Docker Image

After you created your Dockerfile, you can create a Docker image out of it with the `docker build` command. Whereas the Docker container represents a kind of virtual computer, the Docker image represents the hard disk of this computer.

To create a Docker image:

1. In your terminal window, enter:
   `$ docker build -t pylonmipi .`

The `docker build` command consists of three parameters, that are described in the following:

- `-t`: Tags the image with the name `pylonmipi` (you can give it any other name).
- `.`: The "`.`" at the end of the command is the path to the Dockerfile.

	Building this image may take some minutes as a lot of data needs to be downloaded and installed.

After creating the Docker image it must be registered to the docker registry.

To register the Docker image:

1. In your terminal window, enter:
   `$ docker images.`

The following example shows two registered images. One is the Ubuntu image, from which your image was derived, and the newly created pylonmipi image:

```
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
pylonmipi           latest              d45e717c7504        2 minutes ago       1.4GB
ubuntu              21.10               771a596909ab        4 days ago          69.9MB
```

4 Running the Docker Container

In the next step you can start your “virtual computer” by running a Docker container that uses the image. When starting the container, you need to pass a number of parameters, that connect the container with your host system. Even if it is kind of a virtual computer, it still uses the kernel of the host system and you need to connect the host system’s hardware (you plan to use) with your container in order to access them from your container.

To run the Docker container:

1. In your terminal window, enter:
   ```
   $ docker run -it --device /dev/video0:/dev/video0 -v '/dev/bus/usb':'/dev/bus/usb' --
   privileged -e DISPLAY=$DISPLAY -v '/tmp/.X11-unix':'/tmp/.X11-unix' pylonmipi
   ```

The `docker run` command consists of different parameters, that are described in the following:

- `-it`: `i` starts the container in interactive mode and keeps **stdin** open, `t` opens a terminal for the container.
- `--device /dev/video0:/dev/video0`:
  Maps the devices `/dev/video0` of the host system into the container. This is needed to access the Basler MIPI cameras.
- `-v '/dev/bus/usb':'/dev/bus/usb'`:
  Maps the volume `/dev/bus/usb` into the container. This is needed to access the Basler USB3 Vision cameras.
- `--privileged`:
  Grants the container the needed access rights for accessing the host system resources.
- `-e DISPLAY=$DISPLAY`:
  Sets the DISPLAY environment variable of the container to the host system value. This allows for displaying the container’s graphics in the host system.
- `-v '/tmp/.X11-unix':'/tmp/.X11-unix'`:
  Maps the volume /tmp/X11-unix into the container. This is needed to have the required authorization to access to hosts X11 system.

After running the container, you have a “virtual computer” that offers all the comfort of an Ubuntu system, even if the host system might not. As the container still directly uses the kernel and the devices of the host system, you’ll also see no performance decrease for applications running inside the container, compared to running this application in the host system.

The following list describes some useful commands for the container:

- `$ docker ps -a:`
  Gives you a list of all containers running or not running.
- `$ docker stop <container-id>`:
  Stops a running container
- `$ docker restart <container-id>`:
  Restarts the container
- `$ docker attach <container-id>`:
  Finally attaches a terminal to the running container.
- `$ docker rm <container-id>`:
  Will remove the container and all changes you made inside the container will be lost.

For more useful docker commands, see https://docs.docker.com/engine/reference/run/.

Revision History

Document Number	Date	Changes
AW00173001000	25 May 2022	Initial release version of this document.